

MODULE DESIGN

IPS18 — Automated Healthcare Document Processing and Intelligence Platform

Track A — Artificial Intelligence & Machine Learning
B.E. Final Year Capstone Project — Semester 7

1. Introduction

This document describes the modular decomposition of the IPS18 platform. Each module corresponds directly to a Core Feature (CF-01 through CF-08) defined in the Product Requirements Document, so that every functional requirement in the PRD can be traced to a specific, independently designed and testable unit of the system. The Medical Information Extraction module (M4) carries the project's primary research and engineering weight, since it hosts the fine-tuned, noise-adapted Named Entity Recognition model that forms the core technical contribution of the capstone. The remaining modules form the supporting pipeline required to deliver a complete, usable platform around that core.

The design follows a pipeline architecture: a document moves through ingestion, OCR, classification, and extraction in sequence, after which summarization and search operate independently on the extracted output. This separation allows the core extraction module to be developed, trained, and evaluated in isolation — a requirement for producing a rigorous held-out evaluation — while still integrating cleanly into a working end-to-end system.

2. Module Overview

ID	Module	Maps to (PRD)	Core Responsibility
M1	Ingestion Service	CF-01	Accept, validate, and store uploaded documents
M2	OCR & Preprocessing Service	CF-02	Convert scanned/image documents into machine-readable text
M3	Classification Service	CF-03	Identify the type of the uploaded document
M4	Medical Information Extraction Service	CF-04	Fine-tuned NER — extract clinical entities (core module)
M5	Summarization Service	CF-05	Generate faithful, concise document summaries
M6	Search & RAG Q&A Service	CF-06	Semantic search and retrieval-grounded question answering

ID	Module	Maps to (PRD)	Core Responsibility
M7	Document & Processing Management Service	CF-07	Repository, status tracking, and document lifecycle
M8	Source Reference & Verification Service	CF-08	Map AI output back to its exact source location

Table 2.1 — Module summary and PRD traceability

3. Module Descriptions

3.1 M1 — Ingestion Service

The Ingestion Service is the single entry point through which every document enters the platform. Its job is narrow but critical: confirm the upload is valid, store it reliably, and hand it off to the rest of the pipeline without blocking the user.

Responsibilities

- Accept uploads in PDF, JPG, JPEG, and PNG formats
- Validate file type, size, and structural integrity before accepting
- Assign a unique document identifier for traceability across all downstream modules
- Persist the raw file to secure object storage
- Publish a processing-started event to trigger the OCR stage asynchronously

Depends On

Object storage (for raw files) and a message queue (to trigger downstream processing without a synchronous, blocking call).

Output / Data Owned

The `documents` table — id, filename, upload timestamp, uploader identity, current status, and storage path.

3.2 M2 — OCR & Preprocessing Service

This module converts a scanned or image-based document into usable text. It is also where the platform's central engineering challenge lives: OCR output from real, imperfect scans is noisy, and this module is responsible for both producing that text and quantifying how reliable it is.

Responsibilities

- Deskew and denoise the input image prior to recognition
- Run the OCR engine (Tesseract / EasyOCR) to extract raw text
- Preserve logical document structure — headings, paragraphs, and tables — wherever the source layout allows it
- Attach a per-page OCR confidence score, which downstream modules use to flag low-reliability extractions

Depends On

Reads raw files from M1's storage layer.

Output / Data Owned

The ``document_text`` table — raw OCR text, layout metadata, and confidence score, indexed by document ID. This module also hosts, as an internal offline component (not exposed at runtime), the noise-injection pipeline used to construct the noisy-OCR training dataset described in the project's Zeroth Review report.

3.3 M3 — Classification Service

Before extraction can proceed meaningfully, the platform needs to know what kind of document it is looking at — a prescription requires different extraction handling than a laboratory report. This module makes that determination automatically.

Responsibilities

- Predict the document type — prescription, laboratory report, discharge summary, or insurance form — from OCR'd text and layout features
- Attach a confidence score to the predicted category
- Make the classification result available to the extraction module for type-aware processing

Depends On

Reads OCR'd text and layout metadata from M2.

Output / Data Owned

``document_type`` and ``classification_confidence``, attached to the document record.

3.4 M4 — Medical Information Extraction Service (Core Module)

This is the platform's central research and engineering contribution. Where the other modules apply well-established techniques, this module applies a Named Entity Recognition model that has been specifically fine-tuned to remain accurate on OCR-degraded clinical text — the condition real scanned healthcare documents are actually found in. Its design keeps it fully decoupled from the rest of the pipeline so it can be trained, tested, and evaluated independently of summarization or search.

Responsibilities

- Run the fine-tuned ClinicalBERT/BioBERT NER model over OCR'd text to identify patient, diagnosis, medication, dosage, and test-result entities
- Normalize extracted values into a consistent structured format
- Attach a confidence score to each individual extracted entity
- Retain character-level offsets into the source OCR text for every entity, so its exact origin can later be verified

Depends On

Reads OCR'd text (M2) and document type (M3); does not depend on M5 or M6, so it can be evaluated in isolation.

Output / Data Owned

The ``extracted_entities`` table — entity type, value, confidence, and source offset. This is the module against which held-out precision, recall, F1-score, and failure-case analysis are reported.

3.5 M5 — Summarization Service

Once structured information has been extracted, this module produces a short, readable summary of the document — intended to let a clinician grasp the key content of a report in seconds rather than reading it in full.

Responsibilities

- Generate an abstractive summary grounded in both the OCR'd text and the extracted entities
- Run a faithfulness check — comparing the summary against the source content — before it is returned to the user
- Present the summary alongside the original document for side-by-side review

Depends On

Reads OCR'd text (M2) and extracted entities (M4).

Output / Data Owned

The `document\_summary` table, including a stored faithfulness score.

3.6 M6 — Search & RAG Q&A Service

This module allows a user to ask a natural-language question and receive an answer grounded in the actual document corpus, rather than a generic language-model response. It combines semantic retrieval with generation, and every answer is required to carry a reference back to its source.

Responsibilities

- Embed incoming user queries into the same vector space as indexed document content
- Retrieve the most relevant document chunks using vector similarity search
- Construct a retrieval-grounded prompt and generate a natural-language answer
- Return the answer together with the specific document and chunk references that support it

Depends On

A vector store (FAISS/Chroma) populated from processed document content, and an embedding/generation model pair.

Output / Data Owned

Answers are not persisted by default; source references are passed to M8 for on-demand resolution.

3.7 M7 — Document & Processing Management Service

This module does not perform any AI processing itself — it is the operational backbone that tracks every document's progress through the pipeline and gives users visibility and control over their document repository.

Responsibilities

- Maintain a searchable repository of uploaded documents for authorized users
- Track and expose processing status per document — Pending, Processing, Completed, or Failed
- Allow users to view a document alongside its extracted information and summary
- Support document management actions such as viewing, retrieving, and deleting records

Depends On

Reads status signals emitted by M1 through M6 as each stage completes.

Output / Data Owned

Aggregated status and metadata; does not own any AI-generated content itself.

3.8 M8 — Source Reference & Verification Service

Because AI-generated output can be wrong, every piece of information the platform surfaces — an extracted entity, a line in a summary, an answer from the search assistant — must be traceable back to its exact origin in the original document. This module is what makes that verification possible, and it depends on every upstream module carrying offset and provenance metadata forward rather than discarding it.

Responsibilities

- Resolve a stored character offset back to its exact page, section, and highlighted region in the original document
- Serve this resolved location to the frontend so a user can jump directly from an AI-generated value to its source
- Apply uniformly across entity extraction, summarization, and search results, since all three produce offset-tagged output

Depends On

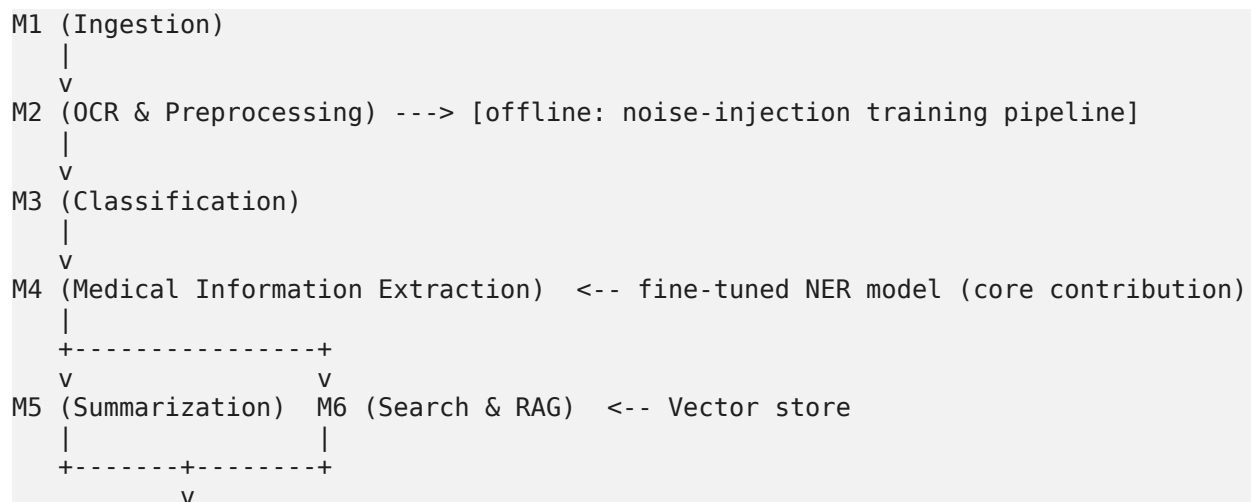
Depends on offset metadata being preserved by M2 (OCR), M4 (extraction), M5 (summarization), and M6 (search) — this is a cross-cutting design requirement rather than a responsibility of M8 alone.

Output / Data Owned

No persistent store of its own; resolves references on demand from data already owned by upstream modules.

4. Module Dependency Flow

The modules are connected in a primarily linear pipeline for ingestion through extraction, after which summarization and search branch off to operate independently on the extracted output. Source verification sits above all content-producing modules, since it depends on offset data from each of them rather than owning a pipeline stage of its own.



M8 (Source Reference & Verification)



M7 (Document & Processing Management) -- tracks status across all stages

4.1 Design Principles Applied

- Loose coupling via events: M1 through M4 are connected through an asynchronous message queue rather than direct synchronous calls, so a slow OCR or extraction stage does not block document ingestion, and each stage can be retried or scaled independently.
- Offset and provenance as a first-class data contract: every module that produces text-derived output is required to carry forward character offsets into the original OCR text, since the Source Reference and Verification module (M8) depends on this being present, not added afterward.
- Isolation of the core research module: M4 takes OCR'd text in and structured entities out, with no dependency on M5 or M6. This lets it be trained and evaluated on held-out data independently of the rest of the platform, which is necessary for a rigorous, honest accuracy assessment.